

PocketSmart AI

Complete Project Documentation

PocketSmart AI

Plan Smart. Spend Smart.

PocketSmart AI is a full-stack GenAI budgeting and recommendation platform for:

- Home interior / home decor
- Party & event planning
- Jewelry shopping

It uses a React + TypeScript frontend, Express + MongoDB backend, and Google's Gemini API for structured recommendation generation. The backend keeps the Gemini API key private and validates every AI response before it reaches the UI.

Current Gemini configuration

The project uses Google's current official JavaScript SDK, `@google/genai`. The default model is `gemini-3.8-flash`, which is a current stable Flash model. The requested historical `gemini-1.5-flash` family is no longer the recommended model for new projects, so the model is configurable through `GEMINI_MODEL`.

Official references:

- Gemini API overview: <https://ai.google.dev/gemini-api/docs>
- Models: <https://ai.google.dev/gemini-api/docs/models>
- Structured output: <https://ai.google.dev/gemini-api/docs/get-started>
- Image understanding: <https://ai.google.dev/gemini-api/docs/generate-content/image-understanding>

Features

- Three adaptive planners
- Reusable budget allocation engine
- Priority-aware requirements
- Structured Gemini JSON output
- Gemini vision input for optional jewelry outfit images
- Demo mode when ``GEMINI_API_KEY`` is missing
- JWT authentication
- MongoDB persistence
- Saved plans and saved recommendations
- What-if budget regeneration

- Recommendation filters and sorting
- Compare up to three recommendations
- Responsive SaaS-style UI
- API validation and centralized error handling
- Rate limiting, Helmet, CORS, upload validation
- Unit/integration tests for core backend behavior
- Docker Compose for local MongoDB

Architecture

Browser

-> React/TypeScript

-> REST API

-> Express controllers

-> services

-> budget engine

-> recommendation engine

-> Gemini service

-> demo service

-> Mongoose models

-> MongoDB

Folder structure

pocketsmart-ai/

■■■■ backend/

■ ■■■■ src/

■ ■ ■■■■ config/

■ ■ ■■■■ controllers/

■ ■ ■■■■ middleware/

■ ■ ■■■■ models/

■ ■ ■■■■ routes/

■ ■ ■■■■ services/

■ ■ ■■■■ utils/

■ ■ ■■■■ validators/

■ ■ ■■■■ app.js

■ ■ ■■■■ server.js

■ ■■■■ tests/

■ ■■■■ .env.example

```
■ ■■■ package.json
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ components/
■ ■ ■■■ hooks/
■ ■ ■■■ layouts/
■ ■ ■■■ pages/
■ ■ ■■■ services/
■ ■ ■■■ types/
■ ■ ■■■ utils/
■ ■ ■■■ App.tsx
■ ■ ■■■ main.tsx
■ ■ ■■■ index.css
■ ■■■ .env.example
■ ■■■ package.json
■ ■■■ tailwind.config.js
■ ■■■ postcss.config.js
■ ■■■ vite.config.ts
■■■ docker-compose.yml
■■■ package.json
```

Requirements

- Node.js 20+
- npm 10+
- MongoDB 7/8, or Docker
- Optional Google Gemini API key

Installation

```
git clone
cd pocketSMART-ai
npm install
npm run install:all
```

If you do not have MongoDB installed:

```
docker compose up -d mongodb
```

Environment setup

Backend

Create:

backend/.env

from:

backend/.env.example

Example:

NODE_ENV=development

PORT=5000

CLIENT_URL=http://localhost:5173

MONGODB_URI=mongodb://127.0.0.1:27017/pocketsmart

JWT_SECRET=replace-with-a-long-random-secret

JWT_EXPIRES_IN=7d

GEMINI_API_KEY=

GEMINI_MODEL=gemini-3.8-flash

DEMO_MODE=true

MAX_UPLOAD_MB=5

Variable meanings:

- ``GEMINI_API_KEY``: Google Gemini API key. Keep it only on the backend.
- ``GEMINI_MODEL``: Gemini model ID. Defaults to ``gemini-3.8-flash``.
- ``MONGODB_URI``: MongoDB connection string.
- ``JWT_SECRET``: secret used to sign authentication tokens.
- ``CLIENT_URL``: allowed frontend origin.
- ``PORT``: backend port.
- ``DEMO_MODE``: when true, missing Gemini credentials use clearly labeled sample recommendations.
- ``MAX_UPLOAD_MB``: maximum image upload size.

Frontend

Create:

frontend/.env

from:

frontend/.env.example

VITE_API_URL=http://localhost:5000/api

Do not put the Gemini API key in frontend environment variables.

Gemini setup

1. Create a Gemini API key in Google AI Studio.
2. Put it in backend/.env as GEMINI_API_KEY.
3. Keep GEMINI_MODEL=gemini-3.8-flash unless you intentionally choose another supported model.
4. Restart the backend.

The app uses Gemini structured output with an explicit JSON schema and validates the resulting object with Zod.

Run in VS Code

Terminal 1:

```
cd pocketsmart-ai
npm install
npm run install:all
docker compose up -d mongodb
```

Terminal 2:

```
cd pocketsmart-ai
npm run dev
```

Open:

<http://localhost:5173>

The backend health endpoint is:

<http://localhost:5000/api/health>

Demo mode

You can run without a Gemini key.

Set:

DEMO_MODE=true

GEMINI_API_KEY=

The UI shows:

> Demo Mode – Prices and recommendations are sample data.

Demo recommendations are intentionally not represented as live shopping data.

Authentication

Register a user from `/register`, then log in. JWTs are stored in browser local storage for this portfolio implementation.

For a production deployment, consider secure HTTP-only cookies, CSRF protection, refresh-token rotation, and a dedicated identity provider.

API documentation

Auth

POST `/api/auth/register`

POST `/api/auth/login`

GET `/api/auth/profile`

AI planners

POST `/api/ai/home-recommendations`

POST `/api/ai/party-recommendations`

POST `/api/ai/jewelry-recommendations`

The AI routes accept JSON. Jewelry accepts an optional multipart image upload using the `image` field.

Plans

POST `/api/plans`

GET `/api/plans`

GET `/api/plans/:id`

PUT `/api/plans/:id`

DELETE `/api/plans/:id`

Saved items

GET `/api/saved-items`

POST `/api/saved-items`

DELETE `/api/saved-items/:id`

User flow

1. User opens a planner.
2. User enters budget and contextual details through a step-by-step form.
3. Frontend validates the form.
4. Frontend sends the request to the backend.
5. Backend authenticates the user when logged in and validates the request.
6. Backend calculates a deterministic budget baseline.
7. Backend calls Gemini with a category-specific prompt and strict JSON schema.
8. Optional jewelry images are passed to Gemini as inline image data.
9. Backend parses and validates Gemini JSON.
10. Recommendation scores are calculated using configurable weights.
11. The result is returned to the frontend.
12. User can filter, sort, compare, save, or persist the plan.
13. Saved plans can be reopened, edited, duplicated, or deleted.

Live commerce integration

This project deliberately does not claim live product prices, stock, ratings, or availability.

`PlatformProvider` in `backend/src/services/platformProvider.js` is an abstraction for future integrations. Until official APIs or permitted affiliate/search integrations are connected, the UI uses clearly labeled search links and estimated prices.

Do not scrape third-party sites in violation of their terms.

Testing

Backend:

```
npm --prefix backend test
```

The tests cover:

- Budget allocation
- Budget validation
- AI JSON parsing
- Recommendation scoring
- Authentication and plan route behavior

Deployment

Frontend

Build:

```
cd frontend
```

```
npm run build
```

Deploy the generated `dist/` directory to a static hosting service.

Set:

```
VITE_API_URL=https://your-api.example.com/api
```

Backend

Install production dependencies and run:

```
npm ci --omit=dev
```

```
npm start
```

Set production values for:

- ``MONGODB_URI``
- ``JWT_SECRET``
- ``GEMINI_API_KEY``
- ``CLIENT_URL``
- ``NODE_ENV=production``

Use HTTPS, a managed MongoDB deployment, secret management, monitoring, backups, and an appropriate reverse proxy.

Production hardening checklist

- Move JWT from local storage to secure HTTP-only cookies.
- Add refresh-token rotation.
- Add email verification and password reset.
- Add account lockout / abuse monitoring.
- Add a managed image storage provider such as Cloudinary.
- Add antivirus/content scanning for uploads.
- Add structured logs and monitoring.
- Add request tracing.
- Add database indexes and backups.
- Add official commerce/service provider APIs.
- Add automated CI/CD.
- Add end-to-end browser tests.

Portfolio notes

This implementation is intentionally designed to demonstrate:

- Full-stack architecture
- GenAI structured-output integration
- Multimodal input
- Deterministic business logic around probabilistic AI
- Secure API-key handling
- Authentication
- Persistence
- Responsive UI
- Production-minded validation and error handling